Runaway Bill

You and Bill compare the current RedFlag DSL to the new version your boss has suggested. The current RedFlag executes blocks immediately. The new RedFlag should execute the setups and the events in a specific order. You start by rewriting the event method:

```
def event(description, &block)
  @events << {:description => description, :condition => block}
end

@events = []
load 'events.rb'
```

The new event method converts the event condition from a block to a Proc. Then it wraps the event's description and the Proc-ified condition in a hash and stores the hash in an array of events. The array is a top-level instance variable (like the ones you read about in *Global Variables and Top-Level Instance Variables*, on page 80), so it can be initialized outside the event method. Finally, the last line loads the file that defines the events. Your plan is to write a setup method similar to the event method, and then write the code that executes events and setups in the correct sequence.

As you ponder your next step, Bill slaps his forehead, mutters something about his wife's birthday party, and runs out the door. Now it's up to you alone. Can you complete the new RedFlag DSL and get the expected output from the test file?

Quiz Solution

You can find many different solutions to this quiz. Here is one:

blocks/redflag_3/redflag.rb

```
def setup(&block)
  @setups << block
end

def event(description, &block)
  @events << {:description => description, :condition => block}
end

@setups = []
@events = []
load 'events.rb'

@events.each do |event|
  @setups.each do |setup|
    setup.call
```